NLLLoss#

https://pytorch.org/docs/stable/generated/torch.nn.modules.loss.NLLLoss.html

Description:

The negative log likelihood loss. It is useful to train a classification problem with C classes. If provided, the optional argument weight should be a 1D Tensor assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set. The input given through a forward call is expected to contain log-probabilities of each class. input has to be a Tensor of size either (minibatch,C) (minibatch, C)(minibatch,C) or (minibatch,C,d1,d2,...,dK)(minibatch, C, d_1, d_2, ..., d_K)(minibatch,C,d1,d2,...,dK) with K≥1K \geq 1K≥1 for the K-dimensional case. The latter is useful for higher dimension inputs, such as computing NLL loss per-pixel for 2D images. Obtaining log-probabilities in a neural network is easily achieved by adding a LogSoftmax layer in the last layer of your network. You may use CrossEntropyLoss instead, if you prefer not to add an extra layer. The target that this loss expects should be a class index in the range [0,C−1] [0, C-1][0,C−1] where C = number of classes; if ignore_index is specified, this loss also accepts this class index (this index may not necessarily be in the class range).

Function Signature

```
class torch.nn.modules.loss.NLLLoss(weight=None,
size_average=None, ignore_index=-100, reduce=None,
reduction='mean')[source]#

Parameters

Shape::

forward(input, target)[source]#

Return type
```

Returns:

Tensor

Code Examples

```
>>> log_softmax = nn.LogSoftmax(dim=1)
>>> loss_fn = nn.NLLLoss()
>>> # input to NLLLoss is of size N x C = 3 x 5
>>> input = torch.randn(3, 5, requires_grad=True)
>>> # each element in target must have 0 <= value < C
>>> target = torch.tensor([1, 0, 4])
>>> loss = loss_fn(log_softmax(input), target)
>>> loss.backward()
>>>
>>>
>>> # 2D loss example (used, for example, with image inputs)
>>> N, C = 5, 4
>>> loss_fn = nn.NLLLoss()
>>> data = torch.randn(N, 16, 10, 10)
>>> conv = nn.Conv2d(16, C, (3, 3))
>>> log_softmax = nn.LogSoftmax(dim=1)
>>> # output of conv forward is of shape [N, C, 8, 8]
>>> output = log_softmax(conv(data))
>>> # each element in target must have 0 <= value < C
>>> target = torch.empty(N, 8, 8, dtype=torch.long).random_(0,
C)
>>> # input to NLLLoss is of size N x C x height (8) x width (8)
>>> loss = loss_fn(output, target)
>>> loss.backward()
```

Detailed Documentation

Documentation

The negative log likelihood loss. It is useful to train a classification problem with C classes.

If provided, the optional argument weight should be a 1D Tensor assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set.

The input given through a forward call is expected to contain log-probabilities of each class. input has to be a Tensor of size either (minibatch,C)(minibatch, C)(minibatch,C) or (minibatch,C,d1,d2,...,dK)(minibatch, C, d_1, d_2, ..., d_K)(minibatch,C,d1,d2,...,dK) with K≥1K \geq 1K≥1 for the K-dimensional case. The latter is useful for higher dimension inputs, such as computing NLL loss per-pixel for 2D images.

Obtaining log-probabilities in a neural network is easily achieved by adding a LogSoftmax layer in the last layer of your network. You may use CrossEntropyLoss instead, if you prefer not to add an extra layer.

The target that this loss expects should be a class index in the range [0,C−1][0, C-1] [0,C−1] where C = number of classes; if ignore_index is specified, this loss also accepts this class index (this index may not necessarily be in the class range).

The unreduced (i.e. with reduction set to 'none') loss can be described as:

where xxx is the input, yyy is the target, www is the weight, and NNN is the batch size. If reduction is not 'none' (default 'mean'), then

weight (Tensor, optional) – a manual rescaling weight given to each class. If given, it has to be a Tensor of size C. Otherwise, it is treated as if having all ones.

size_average (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field size_average is set to False, the losses are

instead summed for each minibatch. Ignored when reduce is False. Default: None

ignore_index (int, optional) – Specifies a target value that is ignored and does not contribute to the input gradient. When size_average is True, the loss is averaged over non-ignored targets.

reduce (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on size_average. When reduce is False, returns a loss per batch element instead and ignores size_average. Default: None

reduction (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the weighted mean of the output is taken, 'sum': the output will be summed. Note: size_average and reduce are in the process of being deprecated, and in the meantime, specifying either of those two args will override reduction. Default: 'mean'

Input: (N,C)(N, C)(N,C) or (C)(C)(C), where C = number of classes, N = batch size, or (N,C,d1,d2,...,dK)(N, C, d_1, d_2, ..., d_K)(N,C,d1,d2,...,dK) with K≥1K \geq 1K≥1 in the case of K-dimensional loss.

Target: (N)(N)(N) or ()()(), where each value is 0≤targets[i]≤C−10 \leq \text{targets}[i] \leq C-10≤targets[i]≤C−1, or (N,d1,d2,...,dK)(N, d_1, d_2, ..., d_K)(N,d1,d2,...,dK) with K≥1K \geq 1K≥1 in the case of K-dimensional loss.

Output: If reduction is 'none', shape (N)(N)(N) or (N,d1,d2,...,dK)(N, d_1, d_2, ..., d_K)(N,d1,d2,...,dK) with K≥1K \geq 1K≥1 in the case of K-dimensional loss. Otherwise, scalar.

Runs the forward pass.